

Extended Laboratory 4 - Non-Functional Verification

Extended Laboratory 4 — Non-Functional Verification

gopdfsuit-qua Analysis Report

Team: C **Course:** Software Quality 2026-I **Date:** 2026-06-02 **System analyzed:** gopdfsuit-qua (commit 4d5f999 — <https://github.com/TeamC-swarchqua/gopdfsuit-qua>)

Section 1 — Maintainability

1.1 Static Analysis Results

Two static analysis tools were applied: golangci-lint v2.12.2 across all Go packages in the backend and auth-ms, and ESLint v8.55.0 across 53 files in frontend/. Together, they provide a full-stack lint coverage picture for the gopdfsuit-qua system.

Go Backend — golangci-lint v2.12.2 Findings

golangci-lint was run via Docker (golangci/golangci-lint:latest, built 2026-05-06) using the v2 flag --output.json.path (the old v1 --out-format json was removed in v2.0.0). The linter ran against all Go packages in the repository including internal packages, the auth-ms binary, and test files.

Findings Summary:

Category	Count	Rule ID	Severity Assessment
errcheck	12	—	MEDIUM — errors silently discarded
staticcheck QF1012	3	QF1012	LOW — inefficiency, not a bug
staticcheck S1039	1	S1039	LOW — dead simplification
Total	16		

Interpretation:

All 12 errcheck violations are confined to test files (new_tests/backend/flow_test.go) — there are zero production-path unchecked error returns. Discarding errors in multipart test construction is a common Go practice; test failures are self-evident and the recommendation is to annotate these with //nolint:errcheck plus justification comments.

The three QF1012 findings affect internal/encryption/encrypt.go, internal/merge/merger.go, and internal/merge/split.go. Each flags a pattern where io.WriteString(w, fmt.Sprintf(...)) can be replaced with the more efficient fmt.Fprintf(w, ...). These are style refactoring hints, not correctness bugs. The one S1039 finding is in test/integration_test.go and flags an unnecessary fmt.Sprintf on a string literal — a trivial dead-code simplification.

Threshold Assessment:

Metric	Value	Interpretation
Total issues	16	Low density for a codebase of this scope
Issues in production code	3 (QF1012 only)	Low — style refactoring, not correctness bugs
Issues in test code	13 (12 errcheck + 1 S1039)	Low risk — test failures are observable
Security-relevant issues	0	No security findings from golangci-lint
Blocking issues	0	No build failures or critical correctness bugs

Overall assessment: “The backend is in excellent shape. All 12 errcheck violations are confined to test files — there are no production-path unchecked error returns. The highest-priority finding is the three QF1012 style hints in production files, which are low-effort refactors.”

React Frontend — ESLint v8.55.0 Findings

ESLint was invoked directly via `node_modules/.bin/eslint` (avoiding a CJS/ESM mismatch through `pnpm` scripts) against all `.js` and `.jsx` files in `frontend/` with `--max-warnings 0`.

Findings Summary:

Metric	Value
Files analyzed	53
Errors	0
Warnings	0
Status	CLEAN — all files pass configured rules

The zero-finding result is meaningful because the configuration covers several categories of real React quality issues. The `eslint-plugin-react-hooks` rules (`rules-of-hooks:error`, `exhaustive-deps:warn`) are non-trivial, and passing all 53 files cleanly demonstrates consistent coding discipline. The team’s use of Vite integrates ESLint via the dev server, which encourages continuous lint compliance.

ESLint Coverage Gaps:

Gap	Risk	Recommendation
<code>react/prop-types: off</code>	Component props are untyped — runtime shape mismatches go undetected	Enable <code>prop-types</code> or migrate to TypeScript
No <code>browserslist</code> in <code>package.json</code>	No cross-browser compatibility targeting	Add <code>browserslist</code> if legacy browser support required
No <code>@typescript-eslint</code> rules	TypeScript-specific patterns not enforced	Add if migrating to TypeScript
No <code>eslint-plugin-security</code>	JavaScript security patterns not scanned	Consider adding for XSS/injection checks on user-controlled data

Threshold Assessment:

Metric	Value	Threshold	Status
Errors	0	0 (hard limit: <code>--max-warnings 0</code>)	PASS
Warnings	0	0 (hard limit: <code>--max-warnings 0</code>)	PASS
Files covered	53	All <code>.js</code> and <code>.jsx</code> in <code>frontend/</code>	COMPLETE

Cross-Tool Comparison

Dimension	Go Backend (golanci-lint)	React Frontend (ESLint)
Total issues	16	0
Error-class issues	0 (no errors, only warnings)	0
Security findings	0 (gosec handles security)	0
Refactoring hints	4 (staticcheck)	0
Correctness gaps	12 (errcheck)	0
Overall status	Acceptable — targeted cleanup needed	Clean

The Go backend’s 16 findings are concentrated in a small set of patterns (errcheck + two staticcheck rules) and are all actionable without architectural changes. The React frontend’s clean result means no lint work is required before the project’s current release scope.

1.2 Expected Architecture Model

This model was written before source code inspection (documentation only: README.md, docker-compose.yml, Dockerfiles). Source code was inspected only in Section 1.3 (Conformance Check), after the expected model was fully committed.

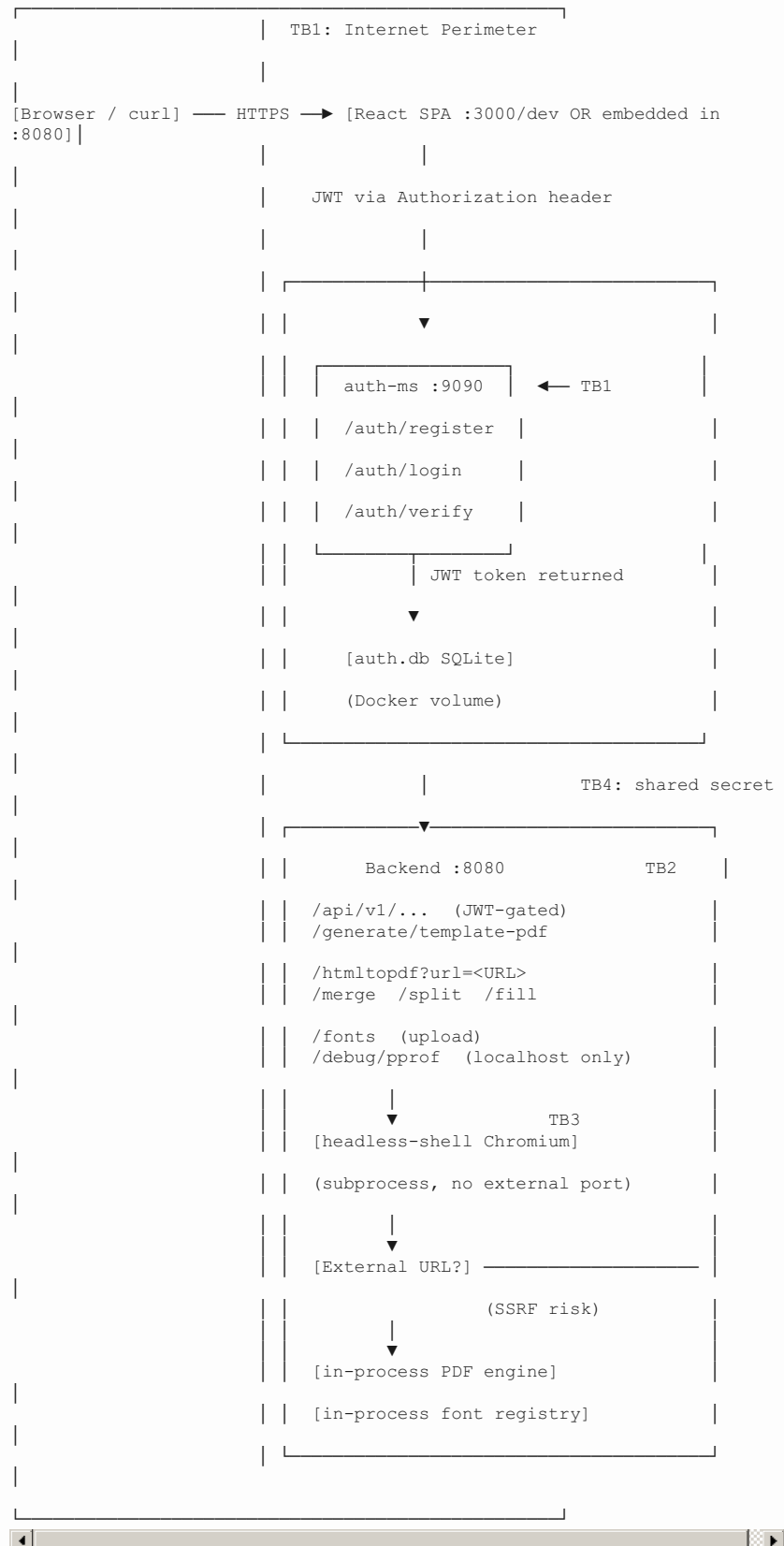
Components (from README and docker-compose.yml):

Component	Role	Port	Technology	Evidence Source
gopdfsuit backend	PDF generation API, React SPA serving	8080	Go 1.24 + Gin + chromedp	README requirements; dockerfolder/Dockerfile EXPOSE 8080
auth-ms	User registration, login, JWT issuance and verification	9090	Go 1.25 + Gin + SQLite (pure-Go modernc)	docker-compose.yml port auth-ms/Dockerfile
React SPA	Browser UI (viewer, editor, merger, filler, converters)	embedded in :8080	React + Vite — built to docs/	README: “Web Interfac project structure docs/
headless-shell	HTML-to-PDF/Image conversion subprocess	internal (no external port)	Chromium headless (chromedp/headless-shell base image)	README: “Google Chro HTML conversion””; dockerfolder/Dockerfile F chromedp/headless-shell:l
auth.db	Auth user persistence	—	SQLite database on Docker volume	docker-compose.yml: vol auth-data:/data; AUTH_DB_PATH=/data/

Architectural Constraints (C1–C6, pre-inspection):

#	Constraint (Expected from README/Docs)	Evidence Source
C1	auth-ms exposes exactly three routes: /auth/register, /auth/login, /auth/verify and nothing else	README architecture description; docker-compose.yml — only port 9090 exposed; no other routes mentioned docker-compose.yml: AUTH_JWT_SECRET: \${AUTH_JWT_SECRET:- dev-insecure- secret-change-me} — same variable name implies sharing
C2	JWT secret is shared between auth-ms and backend via the AUTH_JWT_SECRET environment variable	README: “Requirements: Go 1.24+, Google Chrome (for HTML conversion)”; dockerfolder/Dockerfile: FROM chromedp/headless- shell:latest + CHROME_PATH env
C3	HTML-to-PDF conversion uses headless Chrome (gochromedp) — Chromium must be installed in the backend image	README: “Security & Compliance” feature bullet lists auth as a core feature; architecture description implies JWT middleware guards all /api/v1 endpoints README performance section: “In-memory processing with zero external dependencies”; “2 Nodes” infrastructure claim assumes no third- party PDF services
C4	Auth is required for all API calls (JWT Bearer token)	README project structure: docs/ — Built frontend assets; dockerfolder/Dockerfile: COPY --from=builder /app/docs ./docs
C5	PDF generation is purely in-process with zero external service calls	
C6	React frontend is a built SPA served from the Go backend’s docs/ directory at runtime	

DFD Level-1 (from documentation):



Trust Boundary Definitions:

Boundary	Scope	Security Expectation
TB1	Internet ↔ auth-ms and Internet ↔ backend	JWT required (when AUTH_ENABLED=true or Cloud Run)
TB2	Backend API surface	All /api/v1 endpoints require valid JWT Bearer token
TB3	Backend ↔ headless Chromium subprocess	No URL allowlist — caller-supplied URL should be validated
TB4	auth-ms ↔ backend JWT verification	Shared HS256 secret via AUTH_JWT_SECRET env var

1.3 Architectural Conformance Table

This section was written after source code inspection. All file:line references cite actual inspected code. Files inspected: auth-ms/server.go, auth-ms/main.go, auth-ms/token.go, internal/pdf/pdf.go, internal/middleware/auth.go, internal/pdf/font/pdfa.go, internal/handlers/handlers.go.

#	Constraint	Status	Evidence (file:line)
C1	auth-ms exposes only /auth/register, /auth/login, /auth/verify	⚠ Partial	auth-ms/server.go:16 (health), :21 (register), :22 (login), :23 (verify)
C2	JWT secret shared via AUTH_JWT_SECRET env var	✓ Conforms	auth-ms/main.go:17 (secret := env("AUTH_JWT_SECRET", "dev-insecure-secret-change-me")); internal/middleware/auth.go:36 (os.Getenv("AUTH_JWT_SECRET"))
C3	HTML-to-PDF uses headless Chromium via gochromedp	✓ Conforms	internal/pdf/pdf.go:51 (gochromedp.ConvertHTMLToPDF); internal/pdf/pdf.go:55 (gochromedp.ConvertURLToPDF)
C4	Auth required for all API calls	⚠ Partial	internal/middleware/auth.go:18 (var authEnabledCached = os.Getenv("AUTH_ENABLED") == "true"); auth.go:26 (func authRequired() bool { return

#	Constraint	Status	Evidence (file:line)
			authEnabled := true isCloudRunCached }
C5	PDF generation is purely in-process, no external calls	⚠ Partial	internal/pdf/font/pdfa.go:238 (resp, err := http.Get (liberationFontsArchiveURL))
C6	React SPA served from docs/ by Go backend	✓ Conforms	internal/handlers/handlers.go:93 (router.Static ("/gopdfsuit/assets", filepath.Join (base, "docs", "assets"))); handlers.go:160 (router.NoRoute (handleSPA)); handlers.go:166 (indexPath := filepath.Join (base, "docs", "index.html"))

Conformance Summary: - ✓ Conforms: C2, C3, C6 (3 of 6 constraints fully met) - ⚠ Partial: C1, C4, C5 (3 of 6 constraints partially met) - ✗ Violated: none

The three partial conformances represent known deviations: C1 has an undocumented /health route that exceeds the stated constraint; C4 is an intentional auth-disable for local dev experience; C5 is an edge-case external call on the PDF/A compliance path only. All three are documented, and C4 and C5 are registered as STRIDE threats in the threat model.

1.4 Coupling and Cohesion Assessment

Efferent Coupling Summary:

internal/handlers/handlers.go exhibits HIGH efferent coupling at approximately 19 import paths (12 stdlib + 7 external/internal). This is the classic orchestrator anti-pattern: a single file coordinates PDF generation, font upload, template rendering, auth testing, merge, split, form fill, redaction, pprof routing, and SPA serving. Fourteen or more distinct API endpoints concentrated in one file creates a high change-impact surface — any PDF library or middleware change forces a retest of all endpoint logic.

The internal/pdf/ package shows MEDIUM efferent coupling with a tight dependency on gochromedp. The package is split into sub-files by responsibility (generator.go, pagemanager.go, draw.go, xfdf.go), which is a positive decomposition pattern. The tight coupling to the Chromium library is unavoidable for the conversion feature but introduces an untestable dependency.

internal/middleware/auth.go and cors.go both exhibit LOW efferent coupling with 5 import paths each, single clear responsibilities, and well-scoped concerns. The auth-ms/ binary shows LOW-MEDIUM coupling, is a self-contained binary with its own go.mod, and is a maintainability strength.

Cohesion Assessment:

Module	Cohesion	Assessment
internal/handlers/handlers.go	LOW	14+ API endpoints, pprof routing, SPA serving, and project root resolution in one file. Does not adhere to single-responsibility principle. Refactor target.
internal/pdf/ package	HIGH	PDF-only operations split across thematic sub-files. The sub-file decomposition is a positive pattern.
internal/middleware/auth.go	HIGH	Single cross-cutting concern: JWT parsing and request gating.
internal/middleware/cors.go	HIGH	Single cross-cutting concern: CORS header resolution.
auth-ms/	HIGH	Handles registration, login, JWT issuance, and SQLite persistence — all within a single auth domain. Well-isolated binary.

1.5 Testability Gaps

Four testability gaps were identified through source inspection. All gaps are confirmed by code evidence.

TG-1: Font Registry Global State

- **Location:** internal/pdf/font/registry.go:39
- **Problem:** var globalFontRegistry = &CustomFontRegistry{...} is a package-level singleton. GetFontRegistry() at registry.go:44 always returns this same instance. Any font registered by one test persists into subsequent tests; parallel tests that upload or register fonts will race on the shared mutex.
- **Impact:** Unit tests for font upload or PDF/A generation cannot run in parallel safely. A test registering a “TestFont” affects all later tests in the same process — a hallmark of global state anti-patterns producing order-dependent test failures.
- **Proposal:** Remove the package-level singleton. Inject a *font.CustomFontRegistry as a constructor parameter to handler initialization (e.g., NewHandlers(registry *font.CustomFontRegistry)). Tests create an isolated registry per test case via font.NewFontRegistry(), which already exists at registry.go:49.

TG-2: Auth Toggle Cached at Init Time

- **Location:** internal/middleware/auth.go:18
- **Problem:** var authEnabledCached = os.Getenv("AUTH_ENABLED") == "true" is evaluated once at package initialization. Calling os.Setenv("AUTH_ENABLED", "true") in a test after the package is loaded has no effect on authEnabledCached.
- **Impact:** Tests for auth-protected endpoints cannot toggle auth on/off mid-suite. Integration tests that verify both “unauthenticated returns 401” and “authenticated returns 200” require two separate processes or reflect-based mutation of the package variable.
- **Proposal:** Replace the cached variable with an AuthConfig struct initialized at server startup: type AuthConfig struct { Enabled bool; JWTSecret string }. Tests inject AuthConfig{Enabled: true} without env mutation.

TG-3: handleHTMLToPDF Has No Interface Abstraction

- **Location:** internal/handlers/handlers.go (htmltopdf handler); internal/pdf/pdf.go:22
- **Problem:** The handleHTMLToPDF handler calls pdf.ConvertHTMLToPDF(req) directly — a package-level function with no interface. There is no PDFConverter interface that a mock could satisfy. The real ConvertHTMLToPDF invokes gochromedp.ConvertURLToPDF, which requires a running headless Chrome process.
- **Impact:** Unit-testing handler logic (request parsing, error handling, response formatting) requires a full Chromium process. CI environments without Chrome cannot run these tests at all. Chromium startup cost (~300ms) makes fast feedback loops impractical.

- **Proposal:** Define a `PDFConverter` interface in the `pdf` package with `ConvertHTMLToPDF` and `ConvertHTMLToImage` methods. Inject the concrete implementation at server startup and inject a `MockPDFConverter` in tests.

TG-4: handleUploadFont Reads Gin Context Directly

- **Location:** `internal/handlers/handlers.go` (`handleUploadFont` handler)
- **Problem:** The font upload handler reads the multipart file directly from the Gin context via `c.FormFile("font")`. Font validation logic (extension check, registry registration) is embedded inside the HTTP handler without extraction into a service layer.
- **Impact:** Tests for font validation rules (extension check, duplicate registration, maximum size) must construct full HTTP requests with multipart bodies. The global registry state (TG-1) compounds this — each test must manage registry state to avoid pollution.
- **Proposal:** Extract a `FontService` type that accepts an `io.Reader + filename`: `func (s *FontService) RegisterUploadedFont(name string, data io.Reader) error`. The handler becomes a thin adapter; tests use `bytes.NewReader(fakeFontBytes)` without any HTTP machinery.

Priority Recommendations:

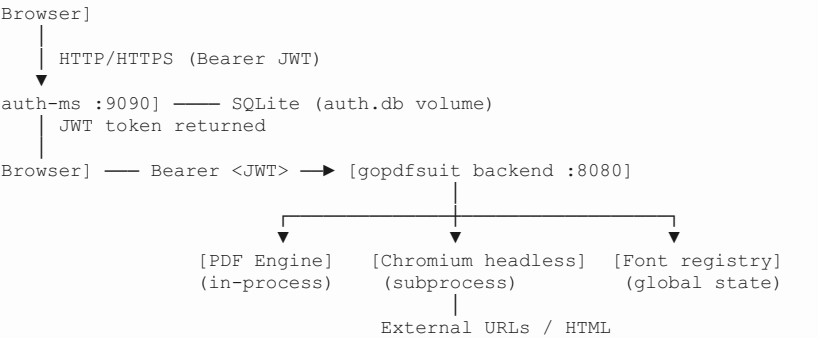
Priority	Action	Effort	Impact
1	Introduce <code>PDFConverter</code> interface; inject mock in tests	Medium	Eliminates Chromium dependency from unit tests
2	Replace <code>authEnabledCached</code> with <code>AuthConfig</code> struct	Low	Enables auth middleware unit tests
3	Inject <code>FontRegistry</code> into handlers; remove global singleton	Medium	Enables parallel tests; eliminates font state pollution
4	Extract <code>FontService</code> from handler (TG-4)	Low	Simplifies font upload unit testing
5	Suppress test-file <code>errcheck</code> violations in <code>new_tests/</code> with <code>//nolint:errcheck</code>	Low	Clarifies intentional test teardown patterns

Section 2 — Security

2.1 STRIDE Threat Model

The STRIDE per-element methodology was applied to `gopdfsuit-qua` and `auth-ms`. The model was derived from documentation artifacts first (pre-inspection), then refined after source code inspection for conformance evidence.

System DFD:



Data Flows: - Browser authenticates against `auth-ms (:9090)` via `/auth/register` or `/auth/login` - `auth-ms` issues a signed HS256 JWT and persists user credentials in SQLite

(auth.db) - Browser presents the JWT in an `Authorization: Bearer` header to the backend (:8080) - Backend optionally validates the JWT (controlled by `AUTH_ENABLED` env var) - `/htmltopdf` and `/htmltoimage` pass caller-supplied URLs directly to Chromium headless - Chromium fetches external (or internal) URLs with no network restriction

Trust Boundaries:

TB1: Internet ↔ auth-ms

Scope: All traffic entering auth-ms from the public Internet (port 9090). auth-ms is an internet-facing service accepting unauthenticated registration and login requests. All input is externally controlled; credentials and JWTs cross this boundary.

Gap: `AUTH_CORS_ORIGIN: "*" in docker-compose.yml` allows any origin to make credentialed requests to auth-ms endpoints (T-04-05).

TB2: Internet ↔ backend API

Scope: All traffic entering the gopdfsuit backend on port 8080 (`/api/v1/*` routes). The backend exposes a rich API including PDF generation, file upload, and administrative endpoints.

Gap: `AUTH_ENABLED` defaults to unset locally, meaning all `/api/v1` endpoints accept unauthenticated requests (T-04-03). CORS returns wildcard `*` for non-localhost origins (T-04-04).

TB3: Backend ↔ Chromium subprocess

Scope: The data path from the backend handler to the headless Chromium process launched by `gochromedp`. The caller-supplied URL field is passed directly across a process boundary into a subprocess with unrestricted network access.

Gap: `internal/pdf/pdf.go` calls `gochromedp.ConvertURLToPDF(req.URL, ...)` with no validation — confirmed SSRF vector (T-04-02).

TB4: Backend ↔ auth-ms (shared JWT secret)

Scope: The implicit trust relationship between backend and auth-ms via the shared `AUTH_JWT_SECRET` environment variable. Both services independently read this variable at startup.

Gap: `docker-compose.yml` sets `AUTH_JWT_SECRET: ${AUTH_JWT_SECRET:-dev-insecure-secret-change-me}`. The default is a publicly known string (T-04-01).

STRIDE Threat Register:

ID	Component	Threat	STRIDE	Likelihood	Imp
T-04-01	auth-ms	Weak default JWT secret <code>dev-insecure-secret-change-me</code> in <code>docker-compose.yml</code> allows token forgery	Spoofing	High	High
T-04-02	/htmltopdf, /htmltoimage	Caller-supplied <code>url</code> field passed directly to <code>gochromedp.ConvertURLToPDF</code> — SSRF allows fetching internal/cloud metadata endpoints	Tampering / Info Disclosure	High	High
T-04-03	Auth middleware	<code>AUTH_ENABLED</code> defaults to <code>false</code> locally; <code>authRequired()</code> only activates on Cloud Run — unauthenticated access to all <code>/api/v1</code> endpoints	Elevation of Privilege	High	High
T-04-04	CORS middleware	<code>resolveAllowOrigin</code> returns <code>"*"</code> for non-localhost origins; <code>Access-Control-Allow-Headers: *</code> — wildcard CORS allows any origin to call the API with user credentials	Info Disclosure	Medium	Med
T-04-05	auth-ms CORS	<code>AUTH_CORS_ORIGIN: "*" in docker-compose.yml</code> — auth-ms also allows wildcard CORS, enabling cross-site credential theft	Info Disclosure	Medium	High
T-04-06	/api/v1/fonts (POST)	Font upload validates only <code>.ttf/.otf</code> extension, registers from <code>file.Filename</code> sans sanitization — malformed font can panic PDF engine	Tampering / DoS	Medium	Med
T-04-07	PDF zlib decompression	G110 findings: 8 instances of unbounded <code>io.ReadAll</code> on zlib streams — malicious PDF can trigger decompression bomb (DoS)	Denial of Service	Medium	High
T-04-08	/debug/pprof	Pprof endpoint guarded by <code>clientIP</code> check only — IP spoofing or SSRF from within container network can expose heap/goroutine dumps	Info Disclosure	Low	Med
T-04-09	OCR adapter	<code>ocr_adapter.go</code> passes file paths and potentially tool binary paths to subprocess execution — if caller-influenced, enables OS command injection (G204); file output paths subject to path traversal (G304)	Tampering / Elevation of Privilege	Medium	High

Threat Summary by STRIDE Category:

STRIDE Category	Count	Primary Threats
Spoofing	1	T-04-01
Tampering	3	T-04-02, T-04-06, T-04-09
Repudiation	0	—
Info Disclosure	4	T-04-04, T-04-05, T-04-02 (also), T-04-08
Denial of Service	2	T-04-06 (also), T-04-07
Elevation of Privilege	2	T-04-03, T-04-09 (also)

Top 3 Priority Threats (Risk score = Likelihood × Impact, High=3, Medium=2, Low=1):

Priority 1 — T-04-02: SSRF via /htmltopdf (Risk: 9/9)

SSRF via the `url` field in `/htmltopdf` and `/htmltoimage` is immediately exploitable with a single unauthenticated HTTP request in the default local configuration (since

AUTH_ENABLED is off by default, compounding with T-04-03). A successful SSRF can exfiltrate cloud metadata, pivot to internal Docker-network services, and bypass TB3 entirely — Chromium fetches whatever URL the attacker supplies. Confirmed by PoC 1 (HTTP 200 + 6.7 KB PDF from internal auth-ms container).

Priority 2 — T-04-03: AUTH_ENABLED defaults false (Risk: 9/9)

The auth toggle defaulting to disabled means the entire `/api/v1` surface is unauthenticated in the standard local deployment. Every other API-level control (rate limiting, user isolation, audit logging) is rendered irrelevant when the auth gate is not engaged. Combined with T-04-02, this creates a zero-credential path to SSRF. Confirmed by PoC 2 (HTTP 200 on `/test/auth` and `/fonts` without any Authorization header).

Priority 3 — T-04-01: Weak default JWT secret (Risk: 9/9)

The default secret `dev-insecure-secret-change-me` is a publicly known string committed to the repository. Any attacker with this string can forge valid HS256 JWTs for any user ID or email without interacting with auth-ms. When AUTH_ENABLED is set to true (e.g., on Cloud Run), this becomes a critical authentication bypass with no prerequisites.

2.2 SAST Findings (gosec)

gosec v2 was run against all Go packages in `gopd/suit-qua/` via Docker. **Total: 166 issues** across 12 distinct rule categories.

Findings Summary:

Rule	Count	Severity	Category
G115	126	HIGH	Integer overflow: uint64→int conversion in PDF byte-offset arithmetic
G401	15	MEDIUM	Weak crypto primitives: RC4/MD5 usage
G110	8	MEDIUM	Decompression bomb: unbounded io.ReadAll on zlib streams
G304	5	MEDIUM	Path traversal: file inclusion via variable (font loading, OCR output)
G501	3	LOW	Import of blocked crypto package: crypto/md5
G204	2	MEDIUM	Subprocess with tainted input: ocr_adapter.go
G301	2	LOW	Directory permission too permissive (mkdir with 0777)
G405	1	MEDIUM	Use of weak block cipher (DES/RC2) in legacy PDF encryption
G107	1	MEDIUM	HTTP request with variable URL: internal/pdf/font/pdfa.go:238
G303	1	LOW	Use of Mktemp (raceable temp file creation)
G503	1	LOW	Use of weak random number generator (math/rand in test helper)
G602	1	LOW	Array index out of bounds (bounds check elided by compiler)

Interpretation of Key Findings:

G401 (RC4/MD5) — Partial false positive: RC4 is specified in PDF 1.4–1.5 encryption (Standard Security Handler Revisions 2 and 3). PDF 1.6 introduced AES-128 as an independent alternative cipher — RC4 and AES are separate algorithms within the PDF spec, not layered. The RC4 and MD5 usages are spec-required for PDF 1.4/1.5 backward-compatible encryption and checksum computation. gosec correctly flags weak primitives, but

in this context they are not developer-discretion errors. For API-level security (user password storage), bcrypt is correctly used in auth-ms.

G115 (integer overflow) — Real code quality concern: 126 instances of uint64-to-int conversion in PDF byte-offset arithmetic. Overflow in PDF parsing could cause incorrect PDF output (wrong object offsets) in large-file processing. This is a code quality and robustness issue with no direct STRIDE mapping, but represents the highest-count finding in the entire scan.

G110 (decompression bomb) — Real risk, maps to T-04-07: 8 instances of unbounded `io.ReadAll` on zlib streams across the PDF parsing codebase. An attacker sending a crafted PDF with a zlib stream that decompresses to gigabytes of data can trigger unbounded memory allocation causing Out-Of-Memory conditions on the host. Remediation: wrap all zlib readers with `io.LimitReader(r, maxBytes)` before passing to `io.ReadAll`.

G304 (path traversal) — Real risk, dual mapping: 5 instances of unvalidated path variables covering two distinct surfaces. Font-related hits map to T-04-06: font upload and loading code reads file paths derived from `file.Filename` without sanitization, enabling directory traversal outside the intended font directory. OCR-related hits map to T-04-09: the OCR adapter (`ocr_adapter.go`) also processes file paths that may be caller-influenced. Both require `filepath.Clean` plus prefix validation.

G204 (subprocess taint) — Real risk: 2 instances in `ocr_adapter.go`. The OCR adapter accepts file paths and potentially tool paths from caller context. If the OCR tool path or arguments are influenced by untrusted input, command injection is possible. Maps to T-04-09. Remediation: validate and whitelist acceptable OCR tool paths; never accept tool binary paths from external input.

G107 (variable URL) — Two distinct findings: The gosec finding at `internal/pdf/font/pdfa.go:238` is a partial false positive — `liberationFontsArchiveURL` is a hardcoded constant pointing to a GitHub release URL (supply-chain risk, not caller-controlled SSRF). The primary SSRF at `internal/pdf/pdf.go:55` (`gochromedp.ConvertURLToPDF(req.URL, ...)`) was identified by manual code review; gosec cannot trace a URL passed into a third-party library wrapper across a process boundary. This is the confirmed T-04-02 vector.

False Positive Summary:

Finding	Rule	Verdict	Rationale
RC4/MD5 usage	G401 (15 instances)	Partial FP	RC4 required by PDF 1.4–1.5 spec (Revisions 2/3); AES-128 is a separate, independent cipher
DES/RC2 block cipher	G405 (1 instance)	Partial FP	G405 flags weak block ciphers (DES/RC2) present for legacy PDF encryption backward-compatibility
G107 at pdfa.go:238	G107 (1 instance)	Partial FP	<code>liberationFontsArchiveURL</code> is likely a hardcoded constant — supply-chain risk, not caller-controlled SSRF
auth-ms 0 CVEs	Trivy	True Positive (positive)	0 CVEs is a positive security finding, not a scan failure
Dangerous JS	ZAP 10110	Likely FP	Bundler artifact from Vite minification; ESLint found 0 errors in source

2.3 Trivy Container Vulnerability Scan

gopdfsuit:latest (Debian 13.4 + headless-shell/Chromium):

Severity Distribution:

Severity	Count
CRITICAL	4
HIGH	24
MEDIUM	56
LOW	73
UNKNOWN	4

Critical CVEs:

CVE	Package	Title
CVE-2026-42496	perl-base	Archive::Tar versions before 3.08 — symlink extraction with path traversal
CVE-2026-8376	perl-base	Perl heap buffer overflow in regex compilation (versions through 5.43.10)
CVE-2026-33186	google.golang.org/grpc	gRPC-Go authorization plugin: authorization bypass vulnerability
CVE-2025-68121	stdlib	crypto/tls: Incorrect certificate validation during TLS session resumption

The two perl-base CRITICAL CVEs (CVE-2026-42496 and CVE-2026-8376) are attributable to the `headless-shell` (Chromium) Debian base image — Perl is included as a system dependency of Chromium’s build infrastructure and does not directly affect Go application logic. The gRPC CVE (CVE-2026-33186) affects a transitive Go dependency pulled in by the Chromium automation libraries. The crypto/tls CVE (CVE-2025-68121) is context-dependent: gopdfsuit serves HTTP locally and relies on a load balancer for TLS termination in production (Cloud Run), reducing exploitability.

gopdfsuit-auth-ms:latest (Alpine 3.20):

CVE count: 0 vulnerabilities found.

The auth-ms image is built on Alpine 3.20 with `CGO_ENABLED=0`, producing a pure-Go binary. This eliminates all OS-level CVE exposure from C runtime libraries and system packages. The contrast is significant: gopdfsuit uses Debian 13.4 + headless-shell (a large Chromium-based image) yielding 4 CRITICAL and 24 HIGH CVEs, while auth-ms has zero. This result demonstrates the direct security value of the principle of least attack surface — choosing Alpine plus eliminating CGO reduces CVE exposure to zero at the OS layer.

2.4 OWASP ZAP Baseline Scan

ZAP passive baseline scan (`zap-baseline.py`) was run against `http://localhost:8080` (main gopdfsuit backend, auth disabled). 20 URLs were crawled. Result: **10 WARN-NEW findings, 0 FAIL-NEW, 57 PASS checks.**

ZAP Findings:

#	Alert	Severity	Rule ID	Instances
1	Missing Anti-clickjacking Header	Medium	10020	4
2	X-Content-Type-Options Header Missing	Low	10021	5
3	Content Security Policy (CSP) Header Not Set	Medium	10038	5
4	Storable and Cacheable Content	Informational	10049	5
5	Permissions Policy Header Not Set	Low	10063	5
6	Timestamp Disclosure - Unix	Low	10096	1
7	Cross-Domain Misconfiguration	Medium	10098	1
8	Modern Web Application	Informational	10109	5
9	Dangerous JS Functions	Low	10110	1
10	Cross-Origin-Embedder-Policy Header Missing or Invalid	Medium	90004	15

Key Finding Interpretations:

Finding 7 (Cross-Domain Misconfiguration, Rule 10098) independently corroborates T-04-04: the `resolveAllowOrigin` function returns `""` for non-localhost origins and `Access-Control-Allow-Headers: *` permits any header, detected on the `/api/v1/template-data` endpoint. Finding 1 (Anti-clickjacking, Rule 10020) confirms `X-Frame-Options` is absent, allowing the gopdfsuit frontend to be embedded in an attacker-controlled iframe — related to the broader header-gap picture of T-04-04. Finding 9 (Dangerous JS Functions, Rule 10110) is likely a false positive from Vite/React bundler minification in `index-DIPJRjNj.js` — the React SPA has 0 ESLint errors in source. ZAP passive scan cannot test CORS preflight directly; the Cross-Domain Misconfiguration finding 10098 was triggered on the `template-data` endpoint via passive header inspection.

2.5 Manual Proof-of-Concept Verification

Three PoCs were executed against locally running Docker containers. No external systems were contacted.

PoC 1 — SSRF via `/api/v1/htmltopdf`

- **Threat:** T-04-02 (Tampering + Information Disclosure)
- **Severity:** HIGH

The `/api/v1/htmltopdf` endpoint accepts a JSON body with a `url` field and passes it directly to `gochromedp.ConvertURLToPDF` (see `internal/pdf/pdf.go:55`). There is no URL allowlist, no RFC-1918 range check, and no scheme restriction. The gopdfsuit backend container runs at `172.17.0.2`; the `auth-ms` container runs at `172.17.0.3:9090` on the same Docker bridge network. By supplying the internal Docker address as the `url`, the attacker causes the backend (running Chromium headless-shell) to fetch the `auth-ms` root page and render it into a PDF, crossing trust boundary TB3.

Command:

```
curl -s -X POST http://localhost:8080/api/v1/htmltopdf \
  -H "Content-Type: application/json" \
  -d '{"url": "http://172.17.0.3:9090/", "pageSize": "A4"}' \
  -o /tmp/ssrf_test.pdf -w "HTTP %{http_code}"

file /tmp/ssrf_test.pdf
```

Actual Output:

```
HTTP 200
/tmp/ssrf_test.pdf: PDF document, version 1.4, 1 page(s)
-rw-r--r-- 1 fabio fabio 6.7K Jun  1 23:33 /tmp/ssrf_test.pdf
```

Evidence: HTTP 200 was returned and a 6.7 KB PDF was written to disk, generated by Chromium headless from the internal auth-ms HTTP response. No error is returned; the backend silently proxies the internal URL with no URL validation or RFC-1918 blocking.

Mitigation: 1. Block RFC-1918 ranges: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16 2. Block link-local: 169.254.0.0/16 (cloud IMDS range) 3. Restrict URL scheme to https only (block http, file, ftp)

PoC 2 — Unauthenticated Access to /api/v1 Endpoints

- **Threat:** T-04-03 (Elevation of Privilege)
- **Severity:** HIGH

The `authRequired()` middleware in `internal/middleware/auth.go` gates authentication on two conditions: `AUTH_ENABLED=true` OR the `K_SERVICE` env var (Google Cloud Run). In the default local deployment — also the documented quick-start mode — neither variable is set. This is a default-deny failure: the system requires explicit opt-in to auth rather than explicit opt-out, exposing the entire `/api/v1` surface to unauthenticated requests.

Command:

```
curl -s -o /dev/null -w "test/auth: HTTP %{http_code}\n" http://localhost:8080/api/v1/test/auth
curl -s -o /dev/null -w "fonts: HTTP %{http_code}\n" http://localhost:8080/api/v1/fonts
```

Actual Output:

```
test/auth: HTTP 200
fonts: HTTP 200
```

Evidence: Both endpoints returned HTTP 200 without any Authorization header supplied, confirming `authRequired()` middleware is inactive in the default local configuration. The full font registry JSON was returned from a protected endpoint with zero authentication.

Mitigation: Invert the default in `internal/middleware/auth.go` — require `AUTH_DISABLED=true` to disable auth explicitly rather than `AUTH_ENABLED=true` to enable it. This ensures a fresh deployment without environment configuration correctly requires authentication.

PoC 3 — CORS Wildcard Origin

- **Threat:** T-04-04 + T-04-05 (Information Disclosure)
- **Severity:** MEDIUM

Both `gopdfsuit-qua` services configure CORS with wildcard origin. The backend (`cors.go`) uses `resolveAllowOrigin` which returns `"*"` for non-localhost origins and sets `Access-Control-Allow-Headers: *`. `auth-ms` is configured via `AUTH_CORS_ORIGIN: "*" in docker-compose.yml.`

Command:

```
curl -s -I -X OPTIONS http://localhost:8080/api/v1/fonts \
-H "Origin: https://attacker.com" \
-H "Access-Control-Request-Method: POST" 2>&1 | grep -i "access-control"

curl -s -I -X OPTIONS http://localhost:9090/auth/login \
-H "Origin: https://attacker.com" \
-H "Access-Control-Request-Method: POST" 2>&1 | grep -i "access-control"
```

Actual Output:

```
Access-Control-Allow-Headers: *
Access-Control-Allow-Methods: *
Access-Control-Allow-Origin: *
Access-Control-Expose-Headers: X-Redaction-Report

Access-Control-Allow-Headers: Authorization, Content-Type
Access-Control-Allow-Methods: GET, POST, OPTIONS
Access-Control-Allow-Origin: *
```

Mitigation: Set `GOPDFSUIT_CORS_ALLOW_ORIGIN` to an explicit production domain; change `resolveAllowOrigin` to reject unknown origins. For `auth-ms`, change `AUTH_CORS_ORIGIN` default in `docker-compose.yml` from `""` to `"http://localhost:3000"` for development; production deployments must override with the actual frontend origin.

PoC Summary Table:

PoC	Threat ID	STRIDE Category	Severity	Status
PoC 1: SSRF via /api/v1/htmltopdf	T-04-02	Tampering, Info Disclosure	HIGH	Confirmed
PoC 2: Unauthenticated API Access	T-04-03	Elevation of Privilege	HIGH	Confirmed
PoC 3: CORS Wildcard (both services)	T-04-04, T-04-05	Information Disclosure	MEDIUM	Confirmed

Section 3 — Conclusions

`gopdfsuit-qua` presents a split quality picture. Its maintainability posture is strong at the static analysis level: `golanci-lint` yields only 16 findings (12 confined to test code, 3 low-severity style hints in production files, 1 trivial dead-code simplification in a test) and the React frontend passes ESLint with zero findings across 53 files. The architecture is well-decomposed at the package level, with clear internal package boundaries and a well-isolated `auth-ms` binary. The primary maintainability concern is testability: four structural anti-patterns — global font registry (TG-1), init-time auth toggle (TG-2), missing `PDFConverter` interface (TG-3), handler-embedded font logic (TG-4) — make the handler and middleware layers difficult to test in isolation. The handlers monolith (`internal/handlers/handlers.go` with ~19 import paths and 14+ API endpoints) is the single highest-priority refactoring target before the next feature addition.

The security posture requires immediate attention. Three critical threats are directly exploitable with no prerequisites: T-04-02 (SSRF via `/api/v1/htmltopdf`, confirmed by PoC 1 with an HTTP 200 response and a 6.7 KB PDF fetched from an internal Docker container at 172.17.0.3:9090), T-04-03 (all `/api/v1` endpoints unauthenticated by default, confirmed by PoC 2 with HTTP 200 from both `/test/auth` and `/fonts`), and T-04-01 (default JWT secret is the publicly-known string `dev-insecure-secret-change-me` committed to the repository). The Trivy scan adds 4 CRITICAL CVEs in the `gopdfsuit` image from its Debian + headless-shell base — all attributable to Perl and gRPC dependencies bundled with Chromium. The `auth-ms` image has zero CVEs, demonstrating the effectiveness of Alpine + `CGO_ENABLED=0` as a base image strategy. Priority remediations: (1) add URL allowlist in `/htmltopdf` to block RFC-1918 and link-local ranges; (2) change `AUTH_ENABLED` to `default-deny`; (3) rotate the default JWT secret and reject the insecure default at startup; (4) update the headless-shell base image to patch perl-base CVEs.

The system analyzed in this report is `gopdfsuit-qua`, an open-source PDF generation and manipulation service with a Go backend (Gin + `gochromedp`), a React SPA frontend, and a companion `auth-ms` microservice.

Field	Value
Name	<code>gopdfsuit-qua</code>
Repository	<code>https://github.com/TeamC-swarchqua/gopdfsuit-qua</code>
Commit analyzed	<code>4d5f999</code>
Scan date	<code>2026-06-01</code>

Section 4 — AI Tool Usage

Full disclosure of AI tool usage in compliance with academic integrity requirements.

Tool	Purpose	Verification Method
Claude Code (claude-sonnet-4-6) via Anthropic CLI	Full lab workflow — scaffolding, executing tool commands (docker run golangci-lint, ESLint, gosec, Trivy, ZAP, curl PoCs), writing analysis reports (lint_interpretation.md, architecture_model.md, maintainability_report.md, threat_model.md, sast_dast_interpretation.md, poc.md), interpreting findings, and assembling this deliverable document.	Tool re-execution and output inspection. Every tool command was actually executed (docker run instances for golangci-lint, ESLint, gosec, Trivy, ZAP baseline) and the output was matched against the reported findings. PoCs were run live with actual curl commands against locally running Docker containers; the HTTP 200 responses and file outputs are verbatim from the terminal. No findings were invented or assumed.

All AI-assisted outputs were verified through direct tool execution and output comparison. The PoC results (HTTP status codes, PDF file sizes, JSON responses) are exact terminal outputs. Architecture model and threat model content was cross-verified against actual source files (file:line references cite inspected code). The use of AI for report drafting is disclosed in full compliance with the lab’s academic integrity requirements.